



gPodder+ Custom Build

User and Maintenance Manual

Practical tasks first, maintenance details later.

Prepared from the code analysis and custom changes discussed in this project.

What is gPodder+?

This is a special project by RobL to update gPodder to be more usable, which is obviously a subjective concept. The big changes made in gPodder+ over standard gPodder include:

- **Podcast episode files follow a Plex-style file naming scheme: <podcast>-SxxEyy-<episode title>**
You can force old episode files to be renamed by first marking all episodes in a podcast as “old” and then marking them as “new.”
- **Podcasts and episodes can be added manually.**
- **Existing podcasts and episodes can be edited manually.**

Quick start

- Use the podcast context menu to manage channel-level actions.
- Double-click an episode to play it.
- Use the manual-entry screens to add or edit podcasts and episodes that do not come from a feed.
- Use “Mark Episodes as Old/New” to toggle episode newness for a podcast; marking as new can also reapply Plex-style filenames.
- Use a high episode limit such as 50,000 or 100,000 if you want a practical near-unlimited archive without relying on the special unlimited value 0.

1. What this custom build does

This build keeps standard gPodder behavior where it still makes sense, but adds several features oriented toward local media management, Plex-friendly filenames, manual data entry, and easier recovery of episodes that were previously deleted or marked old.

- Episodes are stored under a folder for each podcast.
- Downloaded episode filenames can follow a Plex-style pattern based on podcast metadata.
- Season and episode number fields are stored in the database.
- If an episode number is missing from the feed, the publication date can be used as a stable fallback in the filename.
- Double-clicking an episode starts playback.
- Podcast-level actions can toggle episodes between old and new states.
- Deleted episodes that still exist in the database can be undeleted.
- Manual podcast and manual episode entry screens can add and edit records without a live feed.

2. Most useful day-to-day tasks

2.1 Add a manual podcast

1. Open the manual-entry command from the menu location where you wired it into the UI.
2. Choose the option to create a new podcast.
3. Enter the podcast title. This title is important because it drives the podcast folder name and appears in Plex-style filenames.
4. Fill in optional fields such as link, description, and section if you want cleaner metadata.
5. Save the podcast.

Manual podcasts are stored in the database and use an internal synthetic URL. They are not normal downloadable RSS feeds unless you build additional feed-handling logic for them.

2.2 Add a manual episode

6. Select the target podcast.
7. Open the manual-entry command for adding an episode.
8. Enter the title and optional metadata such as publication date, GUID, season, and episode number.
9. Select the media file associated with the episode.
10. Save the episode.

When the episode is saved, the code copies or manages the media file inside gPodder's podcast folder structure and then runs the existing filename logic so the file can match the Plex-style scheme.

2.3 Edit an existing podcast or episode

11. Select the existing podcast or exactly one existing episode.
12. Open the matching edit command from the UI.
13. Update the metadata you want to change.
14. Save the changes.

If you change a podcast title, the podcast rename logic can update the folder name. If you change episode metadata and the episode already has a managed file, the code can rerun filename generation and rename the media file to match the current naming rules.

2.4 Play an episode

Double-clicking an episode row plays the episode. This custom behavior replaces the original double-click action that opened shownotes.

2.5 Toggle episodes between old and new

At the podcast level, the custom context-menu command “Mark Episodes as Old/New” is stateless in the UI, but the code decides what to do internally.

- If the selected podcast currently has new, undownloaded episodes, the action marks applicable episodes old.
- Otherwise it marks episodes new.
- When episodes are marked new, deleted episodes that still exist in the database are undeleted.
- For downloaded episodes, the code can rerun filename generation so the current Plex-style filename is applied.

2.6 Restore episodes that were previously deleted

Episodes that were deleted from disk are usually not removed from the database immediately. Instead they are often retained with a deleted state. Marking such episodes as new undeletes them. This works only for episodes that still exist in the database.

2.7 Use a large episode limit

If you want gPodder to keep very large feeds without aggressive purging of older non-downloaded entries, set a large episode limit such as 50,000 or 100,000. This is often safer than using the special unlimited value 0 because it keeps normal count-based guardrails in place.

3. How podcast folders and filenames work

3.1 Podcast folder naming

The code already prefers the podcast title when creating the podcast download folder. It only falls back to the podcast URL if the title sanitizes to an empty value. Existing podcasts may keep an older folder name until a rename path is triggered.

- New podcasts normally use a sanitized version of the title for the folder.
- Folder names are sanitized so that they are valid on the filesystem.
- Folder names may be made unique if two podcasts would otherwise collide.
- If you rename a podcast, the existing podcast rename logic should be used so files move correctly.

3.2 Episode filename pattern

The target filename pattern is:

<podcast name>_SxxEyy_<episode title>

The practical rules used in this project are:

- Podcast name comes from the channel title.
- Sxx uses the explicit season number when available.
- If season is missing, Sxx falls back to the 4-digit publication year.
- Eyy uses the explicit episode number when available.
- If episode number is missing, Eyy falls back to a publication-date-derived value such as MMDD.
- The file extension still comes from the original media file or MIME/URL-based extension logic.
- The final basename is sanitized before the file is stored.

3.3 Why many episodes show E00 or another fallback

Many podcast feeds do not publish a true episode number. The parser supports iTunes podcast tags inside RSS feeds, but if the feed does not include a usable episode number tag, there is no reliable built-in numeric substitute. In that case a fallback based on publication date is more informative than 00.

4. Understanding season and episode number metadata

The parser does not invent season or episode number values from the title, URL, or filename. It reads them from feed metadata when present.

- Season comes from the feed's season field.
- Episode number comes from the feed's episode-number field.
- If conversion fails or the feed omits those values, the database fields can remain zero or empty-equivalent values depending on your code path.
- Manual-entry screens let you set these values directly for local content.

5. Manual podcasts versus normal feed podcasts

Manual podcasts are database-managed entries, not ordinary network feeds.

- They use a synthetic internal URL so the database has a unique identifier for the podcast.
- That synthetic URL should not be handed to normal feed-fetch code.

- If gPodder tries to update such a channel as if it were a live feed, you can get an error such as “No connection adapters were found for manual://...”.
- The clean fix is to make the channel-update path return immediately for manual podcasts instead of attempting a feed fetch.

6. Episode limits and retention

6.1 What the Preferences value means

The Preferences control for the maximum number of episodes per podcast is not a hard database cap. It is mainly a feed-update and retention setting.

- During feed updates, it limits how far back pagination goes when fetching very large feeds.
- After an update, older non-downloaded episodes can be purged beyond that count.
- Downloaded episodes are generally retained even when older non-downloaded episodes are purged.

6.2 Recommended settings

Target	When to use it	Recommendation
50,000	Large archive with guardrails	Good practical default for very large feeds
100,000	Very large archive with wider headroom	Reasonable if you want more room and can tolerate a larger database
0	Special unlimited mode	Use only if you intentionally want unlimited pagination and no purge behavior

6.3 Files and values to keep aligned

If you choose a high limit, keep these values aligned so the UI and saved preferences agree.

- `gpodderpreferences.ui: adjustment_episode_limit` upper
- `gpodderpreferences.ui: adjustment_episode_limit` value
- `config.py: default_limit.episodes` value
- `Settings.json: persisted_limit.episodes` value for your current profile

For an existing profile, `Settings.json` matters because it overrides the defaults baked into the UI and config files.

7. Common maintenance tasks

7.1 Force existing podcast folders to match the current title

The safe path is to use the existing podcast rename logic. A too-aggressive early-return check can prevent a folder rename when the title is already correct but the folder is still wrong. The improved behavior is to return only when both the title and the computed folder name already match.

7.2 Force downloaded episode files to match the current Plex naming rule

Run the episode filename logic with force-update behavior. That path recalculates the wanted filename and renames the file if the current stored filename no longer matches.

7.3 Upgrade the database schema

When you add columns such as season and number, the database version must be incremented and the migration path must be added. Once those code changes are in place, starting the modified build against the existing database should trigger the upgrade automatically.

7.4 Recover from the manual entry controller startup error

If startup fails with an `AttributeError` saying the main window object has no attribute `manual_entry_controller`, it means `create_actions` ran before the controller was created. The fix is to instantiate the controller before `create_actions` is called in the window setup path that actually runs in your build.

7.5 Avoid accidental feed fetches for manual podcasts

If a manual podcast uses a synthetic URL such as `manual://podcast/...`, the normal update path must short-circuit before feed fetching begins. Otherwise gPodder will pass the synthetic URL to network code and fail.

8. Where the important customizations live

Area	Likely file(s)
GTK main window behavior	gpodder/src/gpodder/gtkui/main.py
Manual-entry controller module	gpodder/src/gpodder/gtkui/manual_entry.py
Podcast and episode model logic	gpodder/src/gpodder/model.py
Database schema and migrations	gpodder/src/gpodder/schema.py
Database purge behavior	gpodder/src/gpodder/dbsqlite.py
Preferences default values	gpodder/src/gpodder/config.py
Preferences UI range and initial value	gpodder/share/gpodder/ui/gtk/gpodderpreferences.ui
Context-menu labels and actions	gpodder/share/gpodder/ui/gtk/menus.ui

8.1 Main UI behaviors changed in this project

- Double-click on an episode plays it.
- Podcast context menus include old/new toggle behavior.
- Manual add/edit actions are attached through a separate controller module.
- Episode rename behavior is triggered when episodes are marked new or edited.

8.2 Model-layer behaviors changed in this project

- Season and episode number are stored in the database and copied from parsed feed metadata.
- Filename generation uses metadata for the basename and standard logic for the extension.
- Manual podcasts bypass ordinary feed updates.
- Podcast folder renames are allowed even when the title is already correct but the folder is not.

9. Suggested workflow when modifying the code

15. Make one small change at a time.
16. Run the GTK UI from the modified source tree, not an installed copy you are not editing.
17. Test the exact user action tied to the change you made.
18. Check both the database effect and the on-disk file effect when renames or manual entry are involved.
19. For filename changes, test with episodes that have season/episode metadata and episodes that do not.
20. For manual podcasts, verify that update commands do not attempt to fetch a feed.
21. After schema changes, verify startup upgrades the database as expected and creates a backup.

10. Troubleshooting guide

10.1 Episode file does not rename

- Make sure the episode already has a managed local file.
- Make sure the rename path uses force-update behavior.
- Check whether the computed target filename is actually different from the current one.
- Check for sanitization or duplicate-name conflict logic that may alter the final filename.

10.2 Podcast folder does not rename

- Use the podcast rename path, not only the save-directory getter.
- Make sure the rename code does not return early just because the title already matches.
- Check for collisions with an existing folder of the same sanitized name.

10.3 Previously deleted episodes do not come back

- The undelete behavior only works for episodes still present in the database.
- If an episode disappeared from both the feed and the local retained database rows, it cannot be undeleted through the simple mark-as-new path.

10.4 Preferences still show the wrong episode limit

- Update the UI upper/value in gpodderpreferences.ui.
- Update the default in config.py.
- Update the saved value in Settings.json for your active profile.

10.5 Manual podcast update throws a network adapter error

- A manual podcast was sent through the normal feed-fetch path.
- Short-circuit channel updates for manual podcasts before feed fetching begins.

11. Reference summary

The most important maintenance idea is to keep the user-visible behavior, the model logic, and the persisted configuration aligned. If the UI says one thing, the defaults say another, and Settings.json says something else, the software will feel inconsistent. The same is true for naming logic: podcast folder naming, episode filename generation, and manual-entry screens should all follow the same underlying rules so that the library remains predictable over time.

11.1 Fast checklist

- Podcast title drives folder naming.
- Episode metadata drives Plex-style basenames.
- Season and episode number are stored in the database.
- Publication date is the best practical fallback when episode number is missing.
- Double-click plays an episode.
- Podcast-level old/new toggling can undelete episodes and rerun renaming.
- Manual podcasts must bypass normal feed updates.
- Large episode limits require updates in the UI, defaults, and Settings.json.